

FOR JAVA DEVELOPERS MOVING TO NESTJS

3 Fields → {}

Spring Boot to NestJS, episode 10: API docs with Swagger

The runtime knows the types. Swagger needs its own metadata. Let the plugin derive it.

The same validated request class documents all three fields in Spring and none in NestJS. It is not that TypeScript erased the types: runtime reflection still reports Number, String, String.

@nestjs/swagger builds schemas from its own Swagger metadata, and reflection alone cannot say which properties a class has or which are required. Write that metadata by hand, or let the CLI plugin generate it at build time.

Versions on this sheet were current on 30 September 2026: @nestjs/swagger 12, Spring Boot 4.1.1 with springdoc 3.1.1.

In scope

- Why a typed, validated NestJS DTO documents nothing by default
- ApiProperty by hand, and the drift it invites, measured
- The CLI plugin: where its fields and its limits come from
- The plugin's file name boundary, and the springdoc naming collision

Not in scope

- File uploads in the docs: next episode
- Swagger UI theming and security schemes

Swagger UI renders the OpenAPI document; the JSON is what was measured.

01 The measurement

SPRING, NO ANNOTATIONS	3 of 3 properties, limits included
NESTJS, NO PLUGIN	<code>{"type":"object","properties":{}}</code> 0 of 3
RUNTIME REFLECTION	design:type Number, String, String
HAND-WRITTEN DRIFT	"USD" 201, "US" 400; document says length 2
PLUGIN, DEFAULT	3 of 3 with minimum 1, minLength/maxLength 3
PLUGIN, SHIM OFF	fields and types stay, every limit vanishes
FILE NOT .dto.ts	0 of 2; ".ts" suffix refused; a specific suffix 2 of 2
SPRINGDOC, SAME NAME	one schema; use-fqn=true gives two

Same three fields on both stacks, nothing added unless stated.

02 The plugin, the documented way

It reads properties, types and requiredness from the TypeScript source at compile time. With `classValidatorShim` on (the default), it also copies class-validator limits into the schema.

```
nest-cli.json

{
  "compilerOptions": {
    "plugins": ["@nestjs/swagger"]
  }
}
```

```
the DTO, no ApiProperty anywhere
```

```
export class CreatePaymentDto {
  @IsInt()
  @IsPositive()
  amountInMinorUnits!: number;

  @IsString()
  @Length(3, 3)
  currency!: string;

  @IsString()
  idempotencyKey!: string;
}
```

"TypeScript's metadata reflection system... can't determine which properties a class consists of, or whether a given property is optional or required."

NestJS documentation, OpenAPI CLI plugin, checked 2026-09-30.

`dtoFileNameSuffix` defaults to `['.dto.ts', '.entity.ts']`

NestJS documentation, OpenAPI CLI plugin, checked 2026-09-30.

03 Why not ApiProperty by hand

The same rule written twice drifts. Nothing fails: the server enforces one rule and the document advertises another.

```
drift.ts
```

```
@IsString()
@Length(3, 3)
@ApiProperty({ minLength: 2, maxLength: 2 })
currency!: string;
```

```
POST "/USD" 201, POST "/US" 400, document: {"type":"string","minLength":2,"maxLength":2}
```

Run on a real machine, 2026-09-30, by `scripts/verify-docs.sh` in the course repository.

04 The Spring side

springdoc infers the schema from the Java classes and Bean Validation with nothing added. Two classes with the same simple name share one schema by default; fully qualified names keep them apart.

```
application.properties
```

```
springdoc.use-fqn=true
```

springdoc.use-fqn: "To enable fully qualified names." Default false.

springdoc-openapi properties, checked 2026-09-30.

Before you ship NestJS API docs

1. Enable the `@nestjs/swagger` plugin in `nest-cli.json`
2. Keep `classValidatorShim` on so the limits come from your validators
3. Name DTO files `*.dto.ts`, or add a specific `dtoFileNameSuffix`
4. Never use `".ts"` as a suffix: the plugin refuses it
5. Avoid restating validator limits in `ApiProperty`
6. In Spring, set `springdoc.use-fqn=true` if simple names can collide



Everything else

-  **Site**
<https://code-with-sam-dev.github.io>
-  **YouTube**
<https://www.youtube.com/@CodewithSam-Dev>
-  **LinkedIn**
<https://www.linkedin.com/company/code-with-sam-dev>
-  **x**
<https://x.com/CodeWithSamDev>
-  **TikTok**
<https://www.tiktok.com/@codewithsamdev>
-  **Instagram**
<https://www.instagram.com/codewithsamdev>
-  **Facebook**
<https://www.facebook.com/codewithsamdev>
-  **GitHub**
<https://github.com/code-with-sam-dev>
-  **Buy me a coffee**
<https://buymeacoffee.com/codewithsam>
-  **Buy me sadza**
<https://buymesadza.com/codewithsam>
- Every measurement, one command: scripts/verify-docs.sh**
<https://github.com/code-with-sam-dev/spring-to-node>
- NestJS: OpenAPI CLI plugin**
<https://docs.nestjs.com/openapi/cli-plugin>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. Swagger is a trademark of SmartBear Software. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-09-30 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.